Script-Controlled ACL: Restricting Project Record Access by Field Value

A practical guide for ServiceNow administrators and developers. Examples use the Project table (`pm_project`); substitute your own table and field names.

1. Overview

ServiceNow's Access Control Lists (ACLs) decide who can read, write, create, and delete records and fields. Most ACLs are simple: a role, a condition, or both. Sometimes that is not enough. When access must depend on the **value of a field on the record itself**, or on the relationship between the user and the record, you need a **script-controlled ACL**.

This guide shows how to restrict access to project records based on a field value. It covers the design, the ACL scripts for each operation, a companion query Business Rule for list performance, testing, troubleshooting, and security best practices.

By the end you will be able to:

- Explain how ACL evaluation works and why a restriction can silently fail.
- Build read, write, create, and delete ACLs that depend on a field value.
- Keep list views fast and counts accurate with a query Business Rule.
- Test and debug the rules with impersonation and Debug Security Rules.
- Extend the pattern to related tables and other field types.

2. The Scenario and Requirements

The organization stores sensitive initiatives in the Project table. A custom choice field, **Confidentiality** (`u_confidentiality`), has the values *Public*, *Internal*, and *Restricted*.

Requirement: a project marked **Restricted** must be accessible only to:

1. The project manager (`project_manager`).
2. The project sponsor (`sponsor`).
3. Members of the project team (records in `pm_project_member`).
4. Users holding the `admin` or `pm_admin` role.

Projects that are not Restricted keep their normal behavior. Everyone who could see them before still can.

Assumptions to verify in your instance

Item	Assumed name	Check
Table	`pm_project`	Your project table may be extended or custom.
Confidentiality field	`u_confidentiality`	Confirm the field name and the choice value `restricted`.
Manager field	`project_manager`	Reference to `sys_user`.
Sponsor field	`sponsor`	Reference to `sys_user`.
Team member table	`pm_project_member`	Use the table your team membership actually lives in.
Privileged roles	`admin`, `pm_admin`	Align with your role model.

3. How ACL Evaluation Works

Most ACL mistakes come from misunderstanding evaluation order, so read this section before writing any script.

3.1 Table-level and field-level ACLs

An ACL is defined by **type**, **operation**, and **name**. The name is either a table (`pm_project`) or a table and field (`pm_project.budget`).

- **Table-level ACL** (field set to `-- None --`): controls whether the record is accessible at all for that operation.
- **Field-level ACL**: controls access to one field on a record the user can already reach.
- **Wildcards**: `pm_project.*` covers all fields; `*.*` and `*` cover everything.

For a read operation, the system checks the table ACL first. If that fails, the user does not see the record. If it passes, each field is then checked against field-level ACLs.

3.2 Specificity

The system looks for the most specific matching ACLs first: a field ACL on the exact table, then field ACLs on parent tables, then wildcard ACLs. The first level that has any matching ACL decides the outcome. It does not keep falling through to broader ACLs once a more specific level has matched.

3.3 Multiple ACLs at the same level are OR-ed

If two ACLs match the same table and operation, access is granted when **either** passes. This is the single most common cause of "my restriction is ignored." A permissive ACL, for example a role-only ACL on `pm_project` read, defeats your script-controlled ACL.

3.4 Within one ACL, everything is AND-ed

A single ACL has up to three checks: the **required roles**, the **condition**, and the **script**. All of them must pass for that ACL to grant access.

3.5 Default deny

When the property `glide.sm.default_mode` is set to deny (recommended), a table with no matching ACL is inaccessible. With allow-by-default, an unprotected table is open. Confirm which mode your instance uses.

3.6 What this means for the design

To restrict by field value you can either:

- **Add a script to the ACL that already grants access**, so one ACL both grants and restricts; or
- **Replace** the permissive ACLs with narrower ones, so no other ACL can grant access to Restricted records.

The safest approach is to ensure the Restricted rule lives inside every ACL that could otherwise grant read access to the table. Adding a *second* ACL does not restrict anything; it can only add access.

4. Prerequisites

- The `security_admin` role. ACL creation requires you to **elevate** (profile menu, *Elevate role*).
- A non-production instance to build and test in.
- Test users: a manager, a sponsor, a team member, an unrelated user with a normal project role, and an admin.
- The Confidentiality field on `pm_project` already created and populated.
- A list of every existing ACL on `pm_project` (see the next section).

5. Step 1: Audit the Existing ACLs

Before creating anything, find out what already grants access.

1. Go to **System Security > Access Control (ACL)**.
2. Filter **Name** *starts with* `pm_project` and **Operation** *is* read.
3. Also search for `pm_project.*`, `task` and `task.*` (because `pm_project` extends `task`), and `*`.
4. For each ACL note the required roles, condition, script, and whether it is active.

You will usually find ACLs inherited from the parent `task` table. These matter because a read ACL on `task` may apply to projects when no more specific ACL exists. Once you create a `pm_project` read ACL, it becomes the controlling ACL for project records and the `task` ACLs no longer apply to them. Make sure your new ACL reproduces any access those parent ACLs were providing, otherwise legitimate users will lose access.

6. Step 2: Create the Read ACL

1. Elevate to `security_admin`.
2. Open **System Security > Access Control (ACL)** and click **New**.
3. Set the fields:
   - **Type:** record
   - **Operation:** read
   - **Name:** Project [pm_project], with the field set to `-- None --`
   - **Active:** checked
   - **Admin overrides:** your choice (see below)
   - **Requires role:** your baseline project role, for example `project_user` (optional)
4. Select **Advanced** and enter the script from the next section.
5. Save.

About "Admin overrides"

When checked, users with the `admin` role pass the ACL regardless of the script. If you want admins to be subject to the same rules, clear it. In most organizations admins keep access, and the script also tests for admin explicitly, so the setting is mostly a matter of clarity.

7. The Read ACL Script

```
(function () {
    // Non-restricted projects: allow (other checks in this ACL still apply)
    if (current.getValue('u_confidentiality') !== 'restricted') {
        return true;
    }

    // Privileged roles
    if (gs.hasRole('admin') || gs.hasRole('pm_admin')) {
        return true;
    }

    var userId = gs.getUserID();

    // Project manager or sponsor
    if (current.getValue('project_manager') == userId ||
        current.getValue('sponsor') == userId) {
        return true;
    }

    // Project team member
    var member = new GlideRecord('pm_project_member');
    member.addQuery('project', current.getUniqueValue());
    member.addQuery('user', userId);
    member.setLimit(1);
    member.query();
    if (member.hasNext()) {
        return true;
    }

    return false;
})();
```

How it works

1. **Cheap checks first.** The field comparison and role checks are in memory and cost almost nothing. The database lookup runs only for Restricted records and only if the earlier checks did not already grant access.
2. `getValue()` **returns strings.** Comparing the string value of a reference field to `gs.getUserID()` is reliable. Avoid comparing a GlideElement object directly.
3. `setLimit(1)` stops the query at the first matching member.
4. **The result.** In an ACL script, the variable `answer` carries the result. Returning a boolean from a self-invoking function works, and so does assigning `answer = true;` or `answer = false;`. Pick one style and use it consistently.

Common script mistakes

- **Returning a string.** `return 'true'` is truthy but is not a boolean and causes confusing behavior. Return real booleans.
- **Forgetting the default.** Every path must end in a return value.
- **Using** `current.field` **comparisons without** `getValue`**.** This can compare objects rather than values.
- **Using client-side APIs.** ACL scripts run on the server; `g_form` and similar objects do not exist.

8. Step 3: Create Write, Delete, and Create ACLs

A read restriction alone is not enough. If a user can still write or delete through another path, the control is incomplete. Create one ACL per operation on `pm_project`.

8.1 Write

Users who can read a Restricted project should not necessarily be able to edit it. A reasonable rule is that the manager, sponsor, and admins can write, while ordinary team members are read-only.

```
(function () {
    if (current.getValue('u_confidentiality') !== 'restricted') {
        return true;
    }
    if (gs.hasRole('admin') || gs.hasRole('pm_admin')) {
        return true;
    }
    var userId = gs.getUserID();
    return current.getValue('project_manager') == userId ||
           current.getValue('sponsor') == userId;
})();
```

8.2 Delete

Deletion is more sensitive than editing. Restrict it to administrators:

```
(function () {
    if (current.getValue('u_confidentiality') !== 'restricted') {
        return true;
    }
    return gs.hasRole('admin') || gs.hasRole('pm_admin');
})();
```

8.3 Create

A create ACL evaluates before the record exists, so there is no saved value to inspect, but `current` holds the values the user is submitting. To stop ordinary users from creating Restricted projects:

```
(function () {
    if (current.getValue('u_confidentiality') !== 'restricted') {
        return true;
    }
    return gs.hasRole('admin') || gs.hasRole('pm_admin');
})();
```

8.4 A subtle point about write

A write ACL checks the record as it is *stored*, not the value the user is trying to set. A user who can edit a Public project could try to change its Confidentiality to Restricted, or a Restricted project to Public, in a single save. Protect the field itself with a field-level write ACL (next section) so only privileged users can change classification.

9. Step 4: Field-Level Protection

Field ACLs let you hide or lock individual fields while the record remains visible.

9.1 Lock the Confidentiality field

Create a write ACL named `pm_project.u_confidentiality`:

- **Operation:** write
- **Requires role:** `pm_admin`

No script is needed. Only users with that role can change the classification.

9.2 Hide sensitive fields from non-members

Suppose budget data should be visible only to the manager, sponsor, and finance users, even on Public projects. Create a read ACL named `pm_project.budget`:

```
(function () {
    if (gs.hasRole('admin') || gs.hasRole('pm_admin') || gs.hasRole('finance_viewer')) {
        return true;
    }
    var userId = gs.getUserID();
    return current.getValue('project_manager') == userId ||
           current.getValue('sponsor') == userId;
})();
```

A user who fails a field read ACL sees the field blank or masked, while the rest of the form works normally.

10. Step 5: Add a Query Business Rule for List Views

ACLs are evaluated **row by row, after the database query returns results**. That has three consequences on list views:

- Performance suffers when the ACL script runs queries for many rows.
- Pagination can look wrong, because a page of 20 rows may display 14 after hidden rows are removed.
- Users may see the notice that records were removed by security constraints.

A **query Business Rule** adds conditions to the database query itself, so restricted rows never leave the database.

10.1 Create the rule

Go to **System Definition > Business Rules** and create a rule:

- **Table:** Project [pm_project]
- **When:** before
- **Query:** checked (Insert, Update, Delete cleared)

```
(function executeRule(current, previous /*null when async*/) {
    if (gs.hasRole('admin') || gs.hasRole('pm_admin')) {
        return;
    }

    var userId = gs.getUserID();

    // Projects where the user is a team member
    var ids = [];
    var m = new GlideRecord('pm_project_member');
    m.addQuery('user', userId);
    m.query();
    while (m.next()) {
        ids.push(m.getValue('project'));
    }

    var qc = current.addQuery('u_confidentiality', '!=', 'restricted');
    qc.addOrCondition('project_manager', userId);
    qc.addOrCondition('sponsor', userId);
    if (ids.length) {
        qc.addOrCondition('sys_id', 'IN', ids.join(','));
    }
})(current, previous);
```

10.2 Team members in the query rule

The rule now includes team members, so their lists and counts match what the read ACL allows. It adds one lookup on `pm_project_member` per list load, so keep that table indexed on `user` and `project`. For users who belong to a very large number of projects, measure the cost; if the ID list grows into the thousands, consider a database view or a Script Include that caches memberships for the session.

10.3 Query rules are not security by themselves

A query Business Rule shapes list and report results, but it does not protect direct record access, some APIs, or every integration path. Treat it as a performance and usability layer on top of the ACLs, never as a replacement.

11. Performance Considerations

- **Order the checks by cost.** Field comparisons, then role checks, then database lookups.
- **Short-circuit early.** Return `true` for the common, unrestricted case immediately.
- **Use** `setLimit(1)` for existence checks.
- **Avoid** `GlideAggregate` **or large loops** in ACL scripts.
- **Cache when appropriate.** If the same lookup repeats in a single transaction, consider storing the user's team memberships in a session-scoped or transaction-scoped variable rather than re-querying per row.
- **Index the membership table.** Make sure `pm_project_member` has an index covering `project` and `user`.
- **Prefer** `hasRole` **over group lookups** when a role would work.
- **Measure.** Use the transaction log and the Debug Security Rules output to see how long ACL evaluation takes on a 100-row list.

12. Related Tables and Data Leakage

Protecting the parent record is not enough if child records expose the same information. Review:

Related table	Risk	Mitigation
`pm_project_task` and `planned_task`	Task names and notes reveal project content.	Add a read ACL that checks the parent project's classification.
`pm_project_member`	Exposes who is on a confidential project.	Restrict by the parent project.
`sys_attachment`	Attachments on restricted projects.	Attachment ACLs inherit from the parent record's read access in many configurations; verify.
`sys_journal_field` (comments, work notes)	Notes visible through activity streams or reports.	Confirm journal access follows the parent record.
Reports and dashboards	Aggregates or lists may bypass your assumptions.	Test with an unrelated user.
Notifications	Emails can leak details to subscribers.	Review notification conditions and recipients.
Project Workspace (SPM)	Workspace pages query related tables directly.	Apply similar ACLs and test through the workspace UI.

A child-table read ACL can reach the parent's classification with dot-walking:

```
(function () {
    var project = current.project.getRefRecord();
    if (!project.isValidRecord()) {
        return true;
    }
    return project.canRead();
})();
```

Calling `canRead()` on the parent reuses the same logic, so you maintain one rule instead of two. Test this carefully, because `canRead()` evaluates the full ACL stack for the current user.

13. Variations for Other Field Types

Group-based access. Allow members of the record's assignment group:

```
return gs.getUser().isMemberOf(current.getValue('assignment_group'));
```

Department or company match. Compare to the user's own record:

```
var userRec = gs.getUser().getRecord();
return current.getValue('department') == userRec.getValue('department');
```

Choice list with several restricted levels. Use a lookup object instead of chained conditions:

```
var needsRole = { restricted: 'pm_admin', secret: 'security_officer' };
var level = current.getValue('u_confidentiality');
if (!needsRole[level]) return true;
return gs.hasRole(needsRole[level]);
```

Reference to a user list field. If a `glide_list` field holds approved users:

```
var list = (current.getValue('u_approved_users') || '').split(',');
return list.indexOf(gs.getUserID()) > -1;
```

Scoped applications. In a scoped app, ACL scripts follow scoped API rules. Some `GlideRecord` and `gs` calls differ, and cross-scope access to the table must be allowed in the application's cross-scope privileges.

14. Testing

14.1 Build a test matrix

User	Role	Relationship	Read	Write	Delete
Manager	project_user	Project manager	Yes	Yes	No
Sponsor	project_user	Sponsor	Yes	Yes	No
Team member	project_user	Member	Yes	No	No
Unrelated user	project_user	None	No	No	No
PM admin	pm_admin	None	Yes	Yes	Yes
Admin	admin	None	Yes	Yes	Yes

Run every row against a Restricted project **and** a Public project. The Public case confirms you did not break existing access.

14.2 Procedure

1. Use **Impersonate User** (profile menu) for each test user.
2. Open the project list. Restricted projects the user should not see must be absent.
3. Open a direct URL such as `pm_project.do?sys_id=<sys_id>` for a Restricted project. An unauthorized user should get a message that the record is not found or access is restricted.
4. Try related paths: open a child task, search globally, open a report, and check the activity stream.
5. Test **write**, **delete**, and **create** with the matrix above.
6. Attempt to change Confidentiality as a non-privileged user.

14.3 Debug Security Rules

Turn on **System Security > Debugging > Debug Security Rules**. When you reload a page, the debug output lists each ACL that was evaluated, whether it passed or failed, and why. Use it to confirm that your ACL ran and to see which other ACL granted access when your restriction does not appear to work. Turn it off afterward; it is verbose.

15. Troubleshooting

Symptom	Likely cause	Fix
Unrelated user can still see Restricted projects	Another ACL (for example `pm_project.*`, a parent `task` ACL, or a role-only ACL) grants access	Use Debug Security Rules to find it; add the restriction to that ACL or remove it
Everyone loses access to all projects	Script error, or the new ACL replaced a parent ACL without reproducing its access	Check the logs for script errors; recreate the baseline access in the new ACL
Manager cannot see own project	Field name or comparison wrong, or manager field is empty	Verify field names; compare with `getValue()`
Lists show fewer rows per page than expected	ACL filtering after the query	Add the query Business Rule
Script works for users but not for integrations	Integration user lacks the role or relationship	Give the integration account an explicit privileged role or an exemption in the script
Team members denied	Wrong member table or field names	Confirm the table and the `project` and `user` columns
Changes do not take effect	ACL cache	Wait briefly, or flush the cache with `cache.do`, then retest
Child tasks leak project data	No child ACLs	Add child-table ACLs as in section 12

16. Security Best Practices

1. **Deny by default.** Keep `glide.sm.default_mode` on deny so that gaps fail closed.
2. **Least privilege.** Grant the minimum role required and avoid broad roles such as `admin` in scripts unless necessary.
3. **Keep logic in one place.** Put shared logic in a Script Include (for example `ProjectAccessUtil.canAccess(project, userId)`) and call it from the ACLs, the query rule, and child-table ACLs. Duplicated logic drifts apart.
4. **Fail closed.** If a lookup throws an error, treat it as no access. Wrap risky code in try/catch and return `false` on failure.
5. **Do not trust the client.** Client scripts and UI policies can hide fields but do not enforce security. Only ACLs and server-side rules do.
6. **Protect the classification field.** If users can change Confidentiality freely, the whole scheme can be bypassed.
7. **Review integrations.** REST APIs, imports, and scheduled jobs often run under accounts with elevated roles. Decide deliberately whether they should see Restricted data.
8. **Document and version.** Capture ACL changes in an update set with a clear name and description, and record the business reason.
9. **Audit periodically.** Review who holds `pm_admin`, and review ACLs after upgrades and new plugin installs.
10. **Test with real roles.** Admins often miss bugs because admin bypasses many checks. Always test as a normal user.

A reusable Script Include

```
var ProjectAccessUtil = Class.create();
ProjectAccessUtil.prototype = {
    initialize: function () {},

    canRead: function (project, userId) {
        try {
            if (project.getValue('u_confidentiality') !== 'restricted') return true;
            if (gs.hasRole('admin') || gs.hasRole('pm_admin')) return true;
            if (project.getValue('project_manager') == userId) return true;
            if (project.getValue('sponsor') == userId) return true;
            return this.isMember(project.getUniqueValue(), userId);
        } catch (e) {
            gs.error('ProjectAccessUtil.canRead failed: ' + e);
            return false;
        }
    },

    isMember: function (projectId, userId) {
        var m = new GlideRecord('pm_project_member');
        m.addQuery('project', projectId);
        m.addQuery('user', userId);
        m.setLimit(1);
        m.query();
        return m.hasNext();
    },

    type: 'ProjectAccessUtil'
};
```

The read ACL then shrinks to one line:

```
(function () {
    return new ProjectAccessUtil().canRead(current, gs.getUserID());
})();
```

17. Deployment Checklist

- ■ Existing ACLs on `pm_project`, `task`, and wildcards have been audited.
- ■ Read, write, delete, and create ACLs are created and active.
- ■ The Confidentiality field has its own write ACL.
- ■ The query Business Rule is in place and tested on large lists.
- ■ Child tables, attachments, notes, reports, and notifications were reviewed.
- ■ The test matrix passed for both Restricted and Public projects.
- ■ Integrations and scheduled jobs were checked.
- ■ Logging added during testing has been removed.
- ■ Changes are captured in an update set and documented.
- ■ Stakeholders approved the access model.

18. Quick Reference

Concept	Key point
ACL result	Booleans or `answer = true/false`
Same-level ACLs	OR-ed together
Checks inside one ACL	Role, condition, and script are AND-ed
Table ACL	Controls whether the record is reachable
Field ACL	Controls individual fields
Query Business Rule	Filters rows in the database for speed and accurate counts
Debugging	Debug Security Rules and Impersonate User
Biggest pitfall	A second permissive ACL silently cancels your restriction

If you share the real table, field names, and who should have access, the scripts in this guide can be tailored to your instance.

Team Members

- Priyadarshini A
- Sai Abirami R
- Manikandan A
- Mithesh V
- Narayana Samy